Assignment 6 – UVM [Extra]

Adding the shift register to the ALSU:

- After completing the ALSU env, you will have 2 environments created, shift reg and ALSU environments.
- Adjust the ALSU to instantiate the shift register to execute the shift and rotate operations.
    - For simplicity, remove the clock and reset from the design and verification environment of the shift register, replace @(negedge clk) in the shift register driver and monitor and replace it with #2. Remove the reset sequence.
    - Datain port of the shift register will be connected to the out port of the ALSU
    - Dataout port of the shift register will be connected to a new internal 6-bit bus that you will create named out_shift_reg
    - Assign the value of out_shift_reg to ALSU out port in shift and rotate valid operations
- Since the shift register is now part of the ALSU, and we have 2 separate environments for both, you will add the 2 environments in the ALSU test, where the ALSU environment will drive the interface of the ALSU, and the shift register environment will be passive and will not drive data to the shift register. The shift register environment will only monitor the interface of the shift register and send the data to its scoreboard and coverage collector.

<u>Steps to add the 2 environments in the ALSU test:</u>

1. Add the shift register and ALSU interfaces in the top module.
2. Pass the interfaces in config db.
3. Connect shift register interface using assign statements. For example:
    - assign shift_if.serial_in = DUT.serial_in_reg; //Assume the ALSU instance name is DUT
4. Add 2 environments to the test.
5. The test will create 2 config objects, one for each environment.
6. Each config object will have 2 variables.
    - Virtual interface
    - Variable named is_active of datatype uvm_active_passive_enum, this is an enum defined in the uvm package that will take the values UVM_ACTIVE or UVM_PASSIVE
7. The test will set the 2 interfaces to each configuration object
8. The test will set the is_active variable of the ALSU configuration object to UVM_ACTIVE and will set the is_active variable of the shift register configuration object to UVM_PASSIVE
9. ALSU will be an active agent, driving the interface while the shift register agent will be a passive agent monitoring only the interface.
10. In each agent, you will retrieve the configuration object in the build phase
    - In the build phase, add if condition checking if the config_obj.is_active variable is equal to UVM_ACTIVE, if yes, then build the driver and sequencer
    - In the connect phase, add if condition checking if the config_obj.is_active variable is equal to UVM_ACTIVE, if yes, then connect the driver and sequencer and also connect the virtual interface of the driver

Kareem Waseem

Deliverables:

1. Take the screenshots of the top module, UVM files, 2 designs, and assertion file of the ALSU.
2. Add the signals of the two interfaces to the wave and make sure that the inputs are driven. Take a screenshot and add it to your PDF.
3. Take screenshots of the code coverage, functional coverage, sequential domain (assertions) coverage for both ALSU and shift register

Kareem Waseem